Week 3 Assignment: Implementing the Parking Charge Calculator

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

April 19, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Design Analysis: The Strategy Pattern 1
- Core Architecture 1
- Concrete Implementations 1
- Reflection on Challenges and Problem-Solving 2
  - Challenge 1: Temporal Data Manipulation 2
  - Problem-Solving: 2
  - Challenge 2: The "Stub" Trap and Bootstrapping 2
  - Problem-Solving: 3
  - Challenge 3: Multi-Factor Logic and TDD 3
  - Problem-Solving: 3
- Design Benefits and Future Scalability 4
  - Decoupling and Testability 4
  - Run-Time Flexibility 4
- Conclusion 5
- References 6

Design Analysis: The Strategy Pattern

Core Architecture

The primary design goal was to adhere to the **Open/Closed Principle**: the system should be open for extension (new pricing rules) but closed for modification (existing transaction logic). To achieve this, I implemented the ParkingChargeStrategy interface. This interface serves as a contract for any pricing algorithm, requiring a calculateCharge method that accepts the base rate, the entry time, and the vehicle's permit.

In this architecture, the ParkingLot class acts as the **Context**. It no longer contains nested if-else or switch statements to determine prices. Instead, it holds a reference to a ParkingChargeStrategy object. This allows each individual parking lot on campus to behave differently, one lot might use a flat rate, while another uses a graduation-day surcharge, simply by injecting a different concrete implementation at construction time or via a setter method.

Concrete Implementations

I developed two distinct strategies to demonstrate multi-factor logic:

1. **TypeBasedStrategy (Standard):** This strategy focuses on vehicle attributes. It uses the baseRate and applies for a 10% discount if the CarType is COMPACT. This satisfies the requirement for a "simple" baseline algorithm while ensuring vehicle type is a measured factor.
2. **WeekendDiscountStrategy (Dynamic):** This more complex strategy utilizes temporal data. It inspects the LocalDateTime to determine if the day is a Saturday or Sunday. If so, it applies a 50% discount. Furthermore, it checks if the car is COMPACT to apply an additional 10% "Eco-Weekend" discount. This demonstrates how multiple factors (Time + Vehicle Type) can be compounded within a single strategy.

Reflection on Challenges and Problem-Solving

Challenge 1: Temporal Data Manipulation

One of the most immediate hurdles was the transition from legacy java.util.Date to the modern java.time API. The initial system used Date, which lacks easy methods for extracting the day of the week or the hour of the day.

Problem-Solving:

I refactored the interface to use LocalDateTime. In the ParkingOffice.park() method, I implemented a conversion layer using Instant and ZoneId.systemDefault(). This ensured that the strategies had access to rich temporal data without forcing the entire legacy database to change its storage format. This decoupling allowed for precise "prime time" logic that would have been error-prone with older libraries.

Challenge 2: The "Stub" Trap and Bootstrapping

During the integration of the Strategy into the ParkingOffice, I encountered repeated UnsupportedOperationException errors. These were caused by IDE-generated stubs in the Money and ParkingOffice classes.

Problem-Solving:

I adopted a "Bottom-Up" debugging approach. Instead of fixing the high-level main method, I moved down to the Money class to ensure its constructors were properly overloaded and encapsulated. I then addressed the ParkingOffice by implementing **Constructor Chaining**. This ensured that the PermitManager and TransactionManager subsystems were always initialized, even when using simplified constructors for testing. This resolved the "bootstrapping" failure where the system would crash before the first command was even issued.

Challenge 3: Multi-Factor Logic and TDD

Determining how to combine a 50% weekend discount with a 10% vehicle discount posed a logical question: should they be additive (60% off) or multiplicative (50% off, then 10% off the remainder)?

Problem-Solving:

I utilized **Test-Driven Development (TDD)** to solve this. I wrote WeekendDiscountStrategyTest before finalizing the logic. By seeing the test fail, I realized that additive discounts could potentially lead to a 0 or negative fee if enough factors were added (e.g., a "Student + Weekend + Compact" discount). To solve this, I implemented a **Sanity Floor** in the code: multiplier = Math.max(0.1, multiplier);. This ensures the business always recovers at least 10% of the base rate, protecting the financial integrity of the system.

Design Benefits and Future Scalability

Decoupling and Testability

The Strategy Pattern significantly improved the testability of the system. Because each strategy is a self-contained class, I could write unit tests for the WeekendDiscountStrategy without ever instantiating a ParkingLot or a Database. This isolation ensures that the pricing math is verified independently of the system's infrastructure.

Run-Time Flexibility

The inclusion of getPricingStrategy() and setPricingStrategy() in the ParkingLot class allows for real-time adjustments. If a lot becomes oversubscribed during a specific event, an administrator can call a command that swaps the strategy to a PrimeTimeSurchargeStrategy without taking the system offline.

Conclusion

The implementation of the Strategy Pattern successfully transformed the Parking Management System from a static application into a flexible business platform. By addressing the challenges of temporal data, overloading constructors for value objects, and using TDD to define discount logic, the resulting code is both robust and maintainable. The system is now prepared for future requirements, such as "Special Day" surcharges or loyalty-based pricing, needing only the addition of new strategy classes rather than a total system overhaul.

References

Martin, Robert C. 2018. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Larman, Craig. 2004. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd ed. Upper Saddle River, NJ: Prentice Hall.